PCPP IT University of Copenhagen, F2026

Exercises week 11

Last update 2026-11-06

These exercises aim to:

- make you reflect on when it is preferable to use a GPU (graphics processing unit) for a computational task instead of a CPU, and vice versa;
- familiarise yourself with CPU vector instructions (SIMD) and how to use them in Java to improve processing speed in numeric and text processing algorithms.

Do this first

The exercises build on the lecture and on Java's documentation of the jdk.incubator.vector package and its classes, especially FloatVector and ShortVector.

All the relevant Java code can be found on the Github page for week 11: (https://github.itu.dk/PCPP/PCPP2026-Public/tree/main/week11). The code for the exercises is in:

`.../week11/code-exercises/week11exercises/....`

Exercise 11.1 *Mandatory*

1. Consider a method `boolean contains(char[] chars, char ch)` that returns true if `ch` appears in `chars` and false otherwise. Do you think it would be difficult to implement on a GPU? Why or why not? Would it be worthwhile to run this method on a GPU instead of the CPU? Consider the amount of computation and the amount of data.
2. Consider a method `int[] histogram(char[] chars)` that returns a histogram of the frequencies of characters in `chars`. The returned array `hist` should have length 256 and `hist[ch]` should be the number of times that the character with code `ch` appears in `chars`, ignoring character codes other than 0–255. Do you think it would be difficult to implement on a GPU? Why or why not? Would it be worthwhile to run this method on a GPU instead of the CPU?
3. Consider different sorting algorithms applied to `int[]`, an array of 32-bit integers, such as:
   - bubblesort
   - quicksort
   - heapsort
   - radixsort

   Which of these will likely be difficult to implement on a graphics processing unit (GPU), and which may be easier to implement? (You are not asked to implement any of them on a GPU, only to think about it).

   Challenging
4. Consider the matrix multiply methods `matrixMultRecur` and `matrixMultRecurAux` in file FloatmatrixMult.java, and consider only the case where arguments `simd` and `mimd` are both false. Basically the idea of this implementation is to subdivide the matrix multiplication problem until the submatrices are small enough to fit in cache or other local memory. Describe the obstacles that could make it hard to implement on a GPU. Are there more plausible ways to achieve the same goal without the obstacles?
5. Consider the matrix multiply method `Strassen`, and consider only the case where argument `simd` is false. Describe the obstacles that could make it hard to implement on a GPU.

Mandatory

Exercise 11.2 The purpose of this exercise and the next one is to explore the use of Java's vector instructions for numeric data processing, specifically for arrays of `float`, that is, 32-bit floating-point numbers.

File VectorExercises.java contains skeleton code used below.

You will need Java version 19 or later (newer is better) and the jdk.incubator.vector package and its FloatVector class. To compile and run from the command line, you must explicitly include the vector operations module:

```
javac --add-modules jdk.incubator.vector YourProgram.java
java --add-modules jdk.incubator.vector YourProgram
```

If you build and run with gradle, and everything works as intended, this should be taken care of by declarations in the code-exercises/week11exercises/app/build.gradle file.

1. In a Java program, define static final field `FLOATSPEC = FloatVector.SPECIES_PREFERRED` of type VectorSpecies<Float> and print `FLOATSPEC.length()`. This is the number of lanes in a FloatVector that your CPU cores can efficiently support. What is the number and how many bits does it represent?

2. Write a non-vectorised method `void arrayScale(float[] A, float scale, float[] C)` that multiplies every element of array `A` with `scale` and stores the results in array `C`. The arrays must have the same length.

   Use the given method `float[] randVec(int n)` to create an array `A` with `n = 1024` random elements in the range $[-1, +1]$. Also allocate an array `C` of the same length. Then measure (using the *Microbenchmarks* techniques) the time to execute `arrayScale(A, 0.42f, C)`. Report the result. The method call involves 1024 floating-point multiplications. Given the measured execution time, how many billion floating-point operations (GFLOP/s) does your computer CPU perform in this case?

3. Write a vectorised method `void arrayScaleSimd(float[] A, float scale, float[] C)` that works like `arrayScale` above but uses FloatVector operations. You will likely need FloatVector's `mul` and `intoArray` methods. Using the same setup as above, measure `arrayScaleSimd(A, 0.42f, C)`. Report the result. How does the speed compare with the previous method?

4. Write a plain non-vectorised method `float arraySum(float[] A)` that returns the sum of all elements of array `A`. Then measure the time to execute `arraySum(A)` for an array of length 1024. Report the result. The method call involves 1024 floating-point additions. Given the measured execution time, how many GFLOP/s does your computer CPU perform in this case?

5. Write a vectorised method `float arraySumSimd(float[] A)` that works like `arraySum` above but uses FloatVector operations. You will likely need FloatVector's `reduceLanes` method and operation VectorOperators.ADD. Using the same setup as above, measure the time to execute `arraySumSimd(A)`. Report the result. How does the speed compare with the previous method? How many GFLOP/s does your computer CPU perform in this case?

Challenging

Exercise 11.3 The purpose of this exercise is to explore the processing of mathematical vectors. Note that "vector" has at least two meanings: (a) a mathematical vector $u = (u_0, u_1, \ldots, u_{n-1})$ that is member of an n-dimensional vector space; and (b) a computer data structure containing 4 or 8 or 16 or ... values of identical type, such as `float`, that are all operated on at the same time. In biology and genetics, "vector" means yet other things.

1. The *cosine similarity* is a measure of the similarity of two mathematical vectors u and v. This is used, for instance, in AI and machine learning where words, or tokens, are represented by mathematical vectors in a high-dimensional (maybe 4096 dimensions) vector space and where cosine similarity reveals how similar the meaning of the words is.

   The cosine similarity is defined as the ratio

$$cosSim(u, v) = \frac{u \cdot v}{|u||v|}$$

where $u \cdot v$ is the dot product (or inner product) of u and v, a number computed as

$$u \cdot v = u_0 v_0 + u_1 v_1 + \cdots + u_{n-1} v_{n-1}$$

when $u = (u_0, u_1, \ldots, u_{n-1})$ and $v = (v_0, v_1, \ldots, v_{n-1})$. The norm $|u|$ of u can be computed as $\sqrt{u \cdot u}$, that is, the square root of the dot product between u and itself.

Define a plain non-vectorised method `float dotProd(float[] u, float[] v)` that computes the dot product of two mathematical vectors, represented as two arrays `u` and `v` of floats. The arrays must have the same length.

Define a plain non-vectorised method `float norm(float[] u)` that computes the norm of `u`.

2. Define a method `float cosSim(float[] u, float[] v)` that computes the cosine similarity of `u` and `v` using the above methods. Check that it returns 1 when `u = v = new float[] { 3.0f, 4.0f }`, and that it returns -1 if you negate `v` to `{ -3.0f, -4.0f }`.

   Create two random vectors `u` and `v` with 1024 elements and measure the time to compute `cosSim(u, v)`. Report it.

3. Define a vectorised method `float dotProdSimd(float[] u, float[] v)` that works like `dotProd` but uses FloatVector operations to compute the dot product. You will likely need FloatVector's method `reduceLanes`. Also define a new `normSimd` method that uses `dotProdSimd`.

   Using these vectorised methods, define a version `cosSimSimd` of `cosSim`. Measure the time to compute cosine similarity with `cosSimSimd(u, v)`. What speedup do you get?

Mandatory

Exercise 11.4 In this exercise you will work with Java vectors for text processing. We shall use the ShortVector type whose lanes are of type `short` to two-byte Unicode characters. (There is no CharVector type).

1. Define a static final field `SHORTSPEC = ShortVector.SPECIES_PREFERRED` of type VectorSpecies-<Short> and print `SHORTSPEC.length()`. This is the number of lanes (that is, `short` values) in a ShortVector that your CPU cores can efficiently support. What is the number and how many bits does it correspond to?

2. The VectorExercises.java file contains a method `textProcessing` that reads a 55 MB file of English text into a string `allText`, and then copies this to an array `chars` of `char[]` whose length is a multiple of 32 (so it will be a multiple of currently known vector lane counts).

   Write a plain non-vectorized Java method `boolean contains(char[] chars, char ch)` that returns true if `ch` appears in `chars`, else false. Measure the time to run `contains(chars, '@')` on the 55 MB text.

3. Write a vectorized Java method `boolean containsSimd(char[] chars, char ch)` that works like `contains` above but uses ShortVector operations. You will likely need the ShortVector method `eq` which produces a VectorMask<Short>, and a VectorMask method to test whether any bit in the mask is true. Measure the time it takes to run `containsSimd(chars, '@')`.

4. Write a plain non-vectorized Java method `int countAZ(char[] chars)` that counts the number of characters in the array that is within the upper case range `'A'` to `'Z'`. Measure the time it takes to run `countAZ(chars)`.

5. Write a vectorized Java method `int countAZSimd(char[] chars)` that works like `countAZ` but uses ShortVector operations. You will likely need to use ShortVector's `compare` method twice, and VectorOperators.LE as well as VectorOperators.GE. These will return VectorMask<Short> results, which support logical operations such as `and` that allow you to combine the results of the comparisons. Measure the time it takes to run `countAZ(chars)`.

6. Write a plain non-vectorized method `String replace(char[] chars, char oldChar, char newChar)` that replaces every occurrence of `oldChar` in `chars` with `newChar`, and then returns a new String created from the resulting `chars` array. Measure the time it takes to run `replace(chars, 'a', '@')` followed by `replace(chars, '@', 'a')` — the idea is that `chars` array should be roughly back to its original contents after both calls have been executed.

7. Write a vectorized Java method `String replaceSimd(char[] chars, char oldChar, char newChar)` that works like `replace` above but uses ShortVector operations. You likely need to use the `compare` method to produce a VectorMask<Short>, then use the mask's `blend` method to set only specific lanes of the vector, then write the vector back into the `chars` array. Measure the time it takes to run `replaceSimd(chars, 'a', '@')` followed by `replaceSimd(chars, '@', 'a')`.

8. Measure the time to run `allText.replace('a', '@').replace('@', 'a')` using Java's built-in String method `replace`. Show the result. Do you think that method uses vector instructions?

9. Write a plain non-vectorized Java method `String toLowerCaseAZ(char[] chars)` that replaces every upper case letter in the range `'A'` to `'Z'` with the corresponding lower case letter in range `'a'` to `'z'`, and then returns a new String created from the resulting `chars` array. To convert an upper case letter `ch` in the described range to the corresponding lower case letter, just add 32. That is, replace `ch` by `ch+32`. Measure the time it takes to run `toLowerCaseAZ(chars)`.

   Note that there is a measurement problem here: After the first execution of `toLowerCaseAZ` in the measurement loop, there are no longer any upper case letters (A–Z) in the array, so subsequent calls to the method change nothing. The method still has to run through the array to discover that, though.

10. Write a vectorized Java method `String toLowerCaseAZSimd(char[] chars)` that works like `toLowerCaseAZ` above but uses ShortVector operations. In addition to ShortVector methods mentioned already, you probably need method `lanewise` in a variant that takes a mask argument. Measure the time it takes to run `toLowerCaseAZSimd(chars)`.

Challenging

11. Imagine a method `replace(char[] chars, String oldStr, String newStr)` that replaces every occurrence of substring `oldStr` in `chars` with string `newStr`. What challenges can you foresee in implementing this method using vector instructions? (You are not expected to implement it).

12. Imagine a fast method `int[] histogram(char[] chars)` that returns a histogram of the frequencies of characters in `chars`. The returned array `hist` should have length 256 and `hist[ch]` should be the number of times the character with code `ch` appears in `chars`. This only for character codes between 0 and 255; all others can be ignored. Can one efficiently implement such a method using vector instructions? What other, perhaps simpler or faster, ways of implementing method `histogram` can you think of? (You are not expected to implement it).